

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)

Институт информационных технологий, математики и механики

Кафедра математического обеспечения и суперкомпьютерных технологий

Направление подготовки: «02.04.02 Фундаментальная информатика и информационные технологии»

Профиль подготовки: «Искусственный интеллект»

**Ответы на вопросы
по дисциплине
«Обучение с подкреплением»**

Выполнил:

студент гр. 3825М1ФИ2

_____ Анненко М.Ю.

Проверил:

преподаватель

_____ Долотов Е.В.

Оглавление

Вопросы из недели 1:	3
1. Базовые понятия RL	3
2. Метод кросс-энтропии (табличный + нейронная сеть)	3
Вопросы из недели 2:	5
1. $V(s)$, $Q(s, a)$ - что это такое?	5
2. Метод Policy iteration	5
Вопросы из недели 3:	7
1. Q-learning, SARSA	7
2. Exploration vs Exploitation	7
Вопросы из недели 4:	9
1. DQN	9
2. Experience replay	9
3. Target Network	10
4. Double DQN	10
Вопросы из недели 6:	12
1. Policy gradient	12
2. REINFORCE	12
3. Actor-Critic	13
Вопросы из недели 7:	14
1. Применение RNN для генерации последовательностей	14
2. Применение RL к дообучению моделей генерации последовательностей	14

Вопросы из недели 1:

1. Базовые понятия RL

Reinforcement Learning (обучение с подкреплением, RL) – это парадигма машинного обучения, в которой агент обучается оптимальной стратегии взаимодействия со средой методом проб и ошибок. Главное отличие RL от обучения с учителем заключается в отсутствии заранее размеченных эталонных ответов: агент самостоятельно генерирует данные, выбирая действия, и получает за них числовую обратную связь (вознаграждение или штраф). При этом агент напрямую влияет на распределение входящих наблюдений, что делает процесс нестационарным и требует баланса между исследованием (exploration) и использованием накопленного опыта (exploitation).

Формально взаимодействие агента и среды описывается Марковским процессом принятия решений (MDP). MDP включает: множество состояний $s \in S$, множество действий $a \in A$, функцию вознаграждения $r \in R$ и динамику среды $P(s_{t+1} | s_t, a_t)$, задающую вероятность перехода в следующее состояние. Ключевым является марковское свойство: будущее состояние зависит только от текущего состояния и выбранного действия, а не от всей предыдущей истории взаимодействий.

Стратегия агента задаётся политикой $\pi(a | s)$, определяющей вероятность выбора действия a в состоянии s . Цель RL-алгоритма – найти такую политику, которая максимизирует математическое ожидание суммарного вознаграждения за сессию: $E_{\pi}[R] \rightarrow \max$, где $R = \sum_t r_t$. На практике это решается итеративно: агент играет несколько эпизодов, собирает траектории, оценивает их качество и обновляет политику в сторону действий, приведших к высоким наградам. RL эффективно применяется там, где нет готовых ответов, но есть чёткий сигнал качества: от показа рекламных баннеров и рекомендательных систем до управления роботами, автономным транспортом и оптимизации игровых стратегий.

2. Метод кросс-энтропии (табличный + нейронная сеть)

Метод кросс-энтропии (Cross-Entropy Method, CEM) – простой, но эффективный эволюционный подход к RL, работающий по циклу: «сыграть → отобрать лучшие → обновить политику». На каждой итерации агент генерирует N полных сессий (эпизодов), вычисляет для каждой суммарное вознаграждение и выбирает M лучших траекторий, называемых элитными сессиями (elite sessions). Политика затем перестраивается так, чтобы с большей вероятностью воспроизводить действия, встречавшиеся в этих элитных траекториях.

Табличный вариант применяется при небольших конечных пространствах состояний. Политика хранится в виде матрицы $\pi(a | s) = A_{s,a}$. Обновление сводится к частотному оцениванию: для каждого состояния s вероятность действия a вычисляется как доля раз, когда агент выбирал a в s среди всех посещений s в элитных сессиях.

$$\text{Формула: } \pi(a | s) = \frac{\sum_{(s_t, a_t) \in \text{Elite}[s_t=s]} [a_t=a]}{\sum_{(s_t, a_t) \in \text{Elite}[s_t=s]} 1}$$

Для задач с большими или непрерывными пространствами используется аппроксимированный СЕМ. Политика параметризуется моделью (чаще всего нейронной сетью) $\pi_w(a | s)$. Вместо явного подсчёта частот веса сети обновляются путём максимизации логарифмического правдоподобия действий из элиты:

$W_{i+1} = W_i + \alpha \nabla \sum_{(s_i, a_i) \in Elite} \log \pi_{W_i}(a_i | s_i)$. На практике это реализуется как обычное обучение с учителем: `nn.fit(elite_states, elite_actions)`, где состояния выступают признаками, а действия из элиты – целевыми метками.

Для непрерывных пространств действий политика моделируется гауссовским распределением $\mathcal{N}(\mu(s), \sigma^2)$, где среднее $\mu(s)$ предсказывается нейросетью, а дисперсия σ^2 фиксируется или обучается отдельно. В этом случае используется регрессор (например, `MLPRegressor`). Для стабилизации обучения применяют буферизацию элит из нескольких прошлых итераций, регуляризацию энтропией (чтобы избежать преждевременной сходимости к субоптимальным действиям) и параллельный сбор данных. Метод прост в отладке и хорошо масштабируется, но может страдать от неэффективного исследования среды при редких или зашумлённых вознаграждениях.

Вопросы из недели 2:

1. $V(s)$, $Q(s, a)$ - что это такое?

В reinforcement learning цели агента формализуются через максимизацию суммарного вознаграждения (return): $G = \sum_t \gamma^t r_t$, где γ – коэффициент дисконтирования, обеспечивающий сходимость бесконечных сумм и моделирующий «затухание» влияния текущих действий на отдалённое будущее. Для оценки качества поведения агента вводятся две фундаментальные функции ценности.

Функция ценности состояния $V^\pi(s)$ (State-value function) определяется как математическое ожидание возврата при условии, что агент начинает в состоянии s и далее следует стратегии π . Интуитивно $V(s)$ отвечает на вопрос: «насколько выгодно находиться в данном состоянии, если продолжать играть согласно текущей политике?». При вычислении $V(s)$ учитывается стохастичность как самой политики $\pi(a | s)$, так и динамики среды $P(s' | s, a)$, поэтому значение усредняется по всем возможным траекториям, исходящим из s .

Функция ценности действия $Q^\pi(s, a)$ (Action-value function) – это ожидаемый возврат при условии, что агент находится в состоянии s , выбирает конкретное действие a , а все последующие шаги выполняет согласно политике π . Ключевое отличие от $V(s)$ заключается в том, что на первом шаге стохастичность политики «отключается»: агент принудительно совершает действие a , и только затем возвращается к вероятностному выбору. Интуиция: «какой выигрыш принесёт конкретный шаг a в состоянии s при последующем следовании π ?».

Между функциями существует прямая связь, выражаемая через уравнения ожидания Беллмана (Bellman expectation equations):

- $Q^\pi(s, a) = E[r + \gamma V^\pi(s') | s, a]$
- $V^\pi(s) = \sum_a \pi(a | s) Q^\pi(s, a)$

Эти соотношения показывают, что ценность состояния есть взвешенная по политике сумма ценностей действий, а ценность действия раскладывается на немедленное вознаграждение и дисконтированную ценность следующего состояния. Векторы V и Q служат основой для сравнения политик, оценки их эффективности и дальнейшего поиска оптимального поведения через уравнения оптимальности Беллмана.

2. Метод Policy iteration

Policy iteration (итерация по политике) – один из классических алгоритмов решения MDP, входящий в общую схему Generalized Policy iteration (GPI). Алгоритм чередует два этапа: оценку текущей политики и её жадное улучшение, повторяя цикл до достижения оптимальности.

- 1) Оценка политики (Policy Evaluation):

На этом этапе решается задача предсказания (prediction): для фиксированной политики π вычисляется её функция ценности $V^\pi(s)$.

Итеративно применяется уравнение ожидания Беллмана:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

Процесс продолжается до сходимости (когда изменение V во всех состояниях становится меньше заданного допуска). Математически это эквивалентно решению системы линейных уравнений с числом неизвестных, равным числу состояний. Оператор Беллмана является сжимающим, что гарантирует сходимость к единственному решению V^π

2) Улучшение политики (Policy Improvement):

Получив V^π , алгоритм вычисляет $Q^\pi(s, a)$ и обновляет политику жадным образом: $\pi'(s) = \arg \max_a Q^\pi(s, a)$. Согласно лемме об улучшении

политики, такая замена гарантирует, что новая политика π' будет не хуже старой: $V^{\pi'}(s) \geq V^\pi(s)$ для

всех s . Интуитивно агент пересматривает свои решения, выбирая в каждом состоянии действие с максимальной ожидаемой отдачей.

Сходимость и свойства: Цикл повторяется, пока политика не перестанет меняться.

Если $\pi' = \pi$, это означает, что политика уже удовлетворяет уравнению оптимальности Беллмана, и найденное решение является оптимальной детерминированной политикой.

Policy iteration требует меньше итераций, чем Value Iteration, но каждый шаг вычислительно тяжелее ($O(|A||S|^2 + |S|^3)$). Метод устойчив к выбору начальной политики и не требует полного прохождения всех состояний на каждой итерации, что делает его надёжным базовым алгоритмом для табличных MDP с известной динамикой.

Вопросы из недели 3:

1. Q-learning, SARSA

Q-learning и SARSA — два базовых model-free алгоритма, основанных на методе временных разностей (Temporal Difference, TD). Оба работают с последовательностями $\langle s, a, r, s' \rangle$ и итеративно обновляют таблицу Q-значений, но различаются способом вычисления целевой оценки, что определяет их принадлежность к off-policy или on-policy классам.

Общий шаг обновления для обоих методов строится как экспоненциальное скользящее среднее: $Q(s_t, a_t) \leftarrow \alpha \tilde{Q}(s_t, a_t) + (1 - \alpha)Q(s_t, a_t)$, где α — скорость обучения. Разница заключается в целевом значении $\tilde{Q}$.

Q-learning вычисляет целевую оценку жадно относительно следующего состояния:

$\tilde{Q}(s, a) = r(s, a) + \gamma \max_a Q(s', a')$. Алгоритм предполагает, что начиная со следующего состояния s' агент будет всегда выбирать действие с максимальным Q-значением, независимо от того, какую политику он использует для сбора данных. Благодаря этому Q-learning является off-policy методом: он способен обучить оптимальную политику π^* , даже если траектории генерируются случайной или исследовательской стратегией.

SARSA (State-Action-Reward-State-Action) использует фактическое следующее действие a' , которое агент на самом деле выбрал согласно текущей политике π : $\tilde{Q}(s, a) = r(s, a) + \gamma Q(s', a')$. Поскольку обновление напрямую зависит от сгенерированного агентом шага, SARSA является on-policy методом. Он оценивает ценность именно той политики, которая управляет агентом в данный момент, включая её исследовательскую составляющую.

На практике это приводит к разному поведению. Q-learning сходится к теоретически оптимальной жадной политике, но может быть «рискованным» при наличии случайных исследовательских шагов. SARSA учитывает вероятность отклонения от жадного выбора и часто находит более устойчивые стратегии (например, обходит опасные зоны). Для уменьшения дисперсии оценок в лекции также упоминается Expected SARSA, где $Q(s', a')$ заменяется на математическое ожидание $E_{a' \sim \pi} Q(s', a')$, что сохраняет on-policy характер, но убирает шум выборочного действия.

2. Exploration vs Exploitation

Дилемма «исследование против использования» возникает из-за того, что агент не обладает априорным знанием среды. Если он всегда выбирает действие с текущим максимальным Q-значением (чистая эксплуатация, exploitation), он быстро застревает в локальном оптимуме и никогда не узнает, существуют ли более выгодные альтернативы.

Exploration (исследование) необходим для сбора новых данных и потенциального обнаружения стратегий с большей суммарной наградой.

Для баланса используются вероятностные стратегии выбора действий:

- ϵ -greedy: с вероятностью ϵ агент выбирает случайное действие, а с вероятностью $1 - \epsilon$ – оптимальное согласно текущей таблице Q . Это простой и эффективный способ гарантировать покрытие пространства действий.
- Softmax (Болцмановское исследование): вероятность выбора действия пропорциональна экспоненте его нормализованного Q -значения:
$$\pi(a | s) = \text{softmax}(Q(s, a)/\tau)$$
. Параметр τ (температура) регулирует степень случайности: при высоком τ выбор почти равномерен, при низком – стремится к жадному.

Для гарантии сходимости к оптимальной политике интенсивность исследования должна постепенно снижаться со временем. В случае ϵ -greedy параметр ϵ плавно уменьшают до нуля (стратегия «greedy in the limit»). Это позволяет агенту активно исследовать среду на ранних этапах, а в финале стабильно использовать найденное оптимальное поведение. Однако в нестационарных средах слишком быстрое затухание ϵ может привести к потере адаптивности.

Влияние выбора стратегии наглядно демонстрируется на примере Cliff World. При обучении Q-learning с ϵ -greedy агент выучит кратчайший путь вдоль края обрыва, но во время обучения из-за случайных шагов (ϵ) будет периодически падать и терять награду. SARSA, напротив, учтёт вероятность случайного отклонения и выучит более безопасный, хотя и более длинный маршрут. Таким образом, exploration не просто дополняет обучение, а напрямую определяет, какую именно политику алгоритм сочтёт оптимальной в реальных условиях взаимодействия.

Вопросы из недели 4:

1. DQN

DQN представляет собой глубокое расширение классического Q-learning, разработанное для работы с большими или непрерывными пространствами состояний, где табличное хранение значений $Q(s, a)$ становится невозможным. Вместо явной таблицы DQN использует нейронную сеть с параметрами θ для аппроксимации Q-функции: $Q(s, a; \theta)$. На практике наиболее эффективной архитектурой считается вариант, когда на вход сети подаётся только состояние s , а на выходе за один прямой проход генерируется вектор Q-значений для всех возможных действий. Это позволяет избежать многократных вычислений для каждой пары (s, a) и ускоряет выбор оптимального действия через $\arg \max$.

Обучение формулируется как задача регрессии с минимизацией среднеквадратичной ошибки (MSE) между предсказанным значением и целевым TD-таргетом:

$$L(\theta) = E \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right)^2 \right].$$
 Градиентный спуск обновляет веса

сети в сторону уменьшения этой ошибки. Для сбора данных применяется ϵ -greedy стратегия, обеспечивающая баланс между исследованием среды и эксплуатацией текущих знаний. Однако наивное применение нейросетей к TD-обучению сталкивается с двумя фундаментальными проблемами: последовательные переходы сильно коррелируют (нарушение предположения об i.i.d. данных), а целевые значения «убегают», поскольку вычисляются той же сетью, параметры которой обновляются. Именно эти ограничения стали прямой мотивацией для введения двух ключевых механизмов стабилизации: Experience Replay и Target Network, без которых базовый Q-learning на глубоких архитектурах практически не сходится.

2. Experience replay

Experience replay – это механизм буферизации и повторного использования прошлого опыта взаимодействия агента со средой, критически важный для стабильности и эффективности глубокого RL. При стандартном online-обучении агент обновляет сеть на каждом шаге, используя только текущий переход $\langle s, a, r, s' \rangle$. Такие данные обладают сильной временной автокорреляцией, что нарушает требования стохастического градиентного спуска к независимости выборок. В результате модель быстро «забывает» ранее посещённые состояния, а кривая обучения характеризуется резкими провалами и нестабильностью.

В методе experience replay все собранные переходы сохраняются в кольцевом буфере (replay buffer) фиксированного размера. На этапе обучения мини-батчи

формируются путём случайной равномерной выборки из этого буфера, а не из текущей траектории. Это эффективно «перемешивает» данные, приближая их распределение к i.i.d., сглаживает градиенты и позволяет многократно переиспользовать редкие или информативные переходы, значительно повышая выборочную эффективность. Важное архитектурное ограничение: метод работает исключительно с off-policy алгоритмами (такими как Q-learning или DQN). Данные в буфере собраны в моменты, когда политика агента была другой (часто более слабой или исследовательской), поэтому on-policy методы (например, SARSA) не могут использовать большой буфер без потери теоретической корректности обновлений. Для on-policy подходов рекомендуется либо не использовать буфер, либо компенсировать его малый размер параллельными игровыми сессиями.

3. Target Network

Target Network (целевая сеть) — это архитектурный приём, решающий проблему «движущейся цели» (moving target) и высокой автокорреляции при обучении DQN. В базовой формуле TD-ошибки целевое значение $r + \gamma \max_{a'} Q(s', a'; \theta)$ вычисляется той же сетью с параметрами θ , которые обновляются на каждом шаге. Из-за этого возникает нежелательная автокорреляция: небольшое изменение весов сразу меняет цель для следующего обновления, градиенты становятся нестабильными, и процесс обучения часто расходится.

Идея заключается во введении второй копии сети с параметрами θ^- , веса которой фиксируются на протяжении нескольких итераций и обновляются редко. Целевое значение теперь вычисляется как $r + \gamma \max_{a'} Q(s', a'; \theta^-)$, что делает его константой на промежутке между обновлениями и стабилизирует оптимизацию. Существуют два стандартных способа синхронизации θ^- с основной сетью:

- 1) Жёсткое обновление (Hard update): раз в N тренировочных шагов (например, каждые 5000 шагов в оригинальной работе по Atari) веса целевой сети полностью копируются из основной: $\theta^- \leftarrow \theta$.
- 2) Мягкое обновление (Soft update): на каждом шаге применяется экспоненциальное сглаживание: $\theta^- \leftarrow (1 - \alpha)\theta^- + \alpha\theta$. Мягкое обновление обеспечивает более плавную и непрерывную стабилизацию цели, что особенно эффективно в задачах с непрерывным управлением и чувствительными гиперпараметрами. Использование таргет-сети стало стандартом для всех последующих off-policy алгоритмов глубокого RL.

4. Double DQN

Double DQN был предложен для устранения систематической ошибки переоценки, присущей классическому Q-learning и базовому DQN. Эта ошибка

возникает из-за оператора $\max$ в целевой функции: поскольку Q -значения содержат случайный шум аппроксимации или выборки, математическое ожидание максимума всегда больше или равно максимуму математических ожиданий ($E[\max Q] \geq \max E[Q]$). В результате агент последовательно переоценивает ценность действий, особенно в состояниях с высокой дисперсией наград, что может приводить к субоптимальным стратегиям и нестабильной сходимости.

Алгоритм решает эту проблему, разделяя этапы выбора действия и оценки его ценности. В оригинальном Double Q-learning использовались две независимо обучаемые сети, но в Deep RL-версии (DeepMind, 2015) для этого эффективно переиспользуются уже имеющиеся основная и целевая сети DQN. Формула таргета модифицируется следующим образом: действие выбирается основной сетью $a^* = \arg \max_{a'} Q(s', a'; \theta)$, а его ценность оценивается целевой сетью $Q(s', a^*; \theta^-)$. Итоговый таргет принимает вид: $y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$. Такое разделение значительно снижает положительное смещение оценок, делает кривую обучения более гладкой и, согласно экспериментальным данным, улучшает как медианные, так и средние показатели производительности на сложных средах (Atari) по сравнению с базовым DQN, не требуя при этом введения дополнительных архитектур или вычислительных затрат.

Вопросы из недели 6:

1. Policy gradient

Policy gradient – это семейство методов обучения с подкреплением, которые оптимизируют политику $\pi_\theta(a | s)$ напрямую, без промежуточного этапа оценки функции ценности. В отличие от value-based подходов (например, Q-learning), где агент сначала решает сложную задачу предсказания, а затем извлекает жадную стратегию, Policy gradient максимизирует математическое ожидание суммарного вознаграждения $J(\theta) = E[R]$ непосредственно через градиентный подъём по параметрам θ . Метод работает со стохастическими политиками, что автоматически обеспечивает исследование среды (exploration) и позволяет естественным образом работать с непрерывными пространствами действий (через параметрические распределения, например, гауссово или категориальное).

Ключевая математическая идея метода — логарифмический производный трюк (log-derivative trick). Поскольку градиент ожидания нельзя вычислить аналитически из-за зависимости распределения траекторий от параметров, используется тождество $\nabla \pi_\theta(a | s) = \pi_\theta(a | s) \nabla \log \pi_\theta(a | s)$. Подстановка этого соотношения в производную целевой функции преобразует её в математическое ожидание: $\nabla J(\theta) = E_{s \sim d^\pi, a \sim \pi_\theta} [\nabla \log \pi_\theta(a | s) \cdot Q(s, a)]$. Интуитивно это означает, что мы увеличиваем вероятность действий, приводящих к высоким возвратам, и уменьшаем вероятность неудачных действий, масштабируя обновление логарифмической производной политики. Такой подход даёт несмещённую оценку градиента, вычисляемую через выборочное усреднение по сгенерированным траекториям, и избегает проблем с движущейся целью, характерных для TD-методов.

2. REINFORCE

REINFORCE — это базовый алгоритм Policy Gradient, использующий метод Монте-Карло для оценки градиента. Алгоритм работает строго on-policy и требует полных эпизодов взаимодействия со средой. На каждой итерации агент генерирует N траекторий под текущей политикой π_θ , для каждого шага вычисляет полный дисконтированный возврат $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$, который служит несмещённой оценкой $Q(s_t, a_t)$. Градиент оценивается как $\nabla J \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla \log \pi_\theta(a_t | s_t) \cdot G_t$, после чего параметры обновляются в направлении градиента с шагом α .

Главная проблема REINFORCE — высокая дисперсия оценок. Поскольку G_t агрегирует все будущие награды эпизода, шум среды и стохастичность политики накапливаются, делая обновления нестабильными, особенно в длинных сессиях. В лекции показано, что дисперсию можно радикально снизить без смещения градиента с помощью базисной функции (baseline) $b(s)$. Добавление вычитаемого слагаемого $b(s)$,

не зависящего от действия a , не меняет математическое ожидание градиента, так как $E[\nabla \log \pi_{\theta}(a | s) b(s)] = 0$, но уменьшает разброс значений, если $b(s)$ коррелирует с $Q(s, a)$. На практике в качестве baseline часто используют оценку состояния $V(s)$, что логически подводит к архитектуре Actor-Critic и вводит понятие преимущества (Advantage) $A(s, a) = Q(s, a) - V(s)$.

3. Actor-Critic

Actor-Critic (A2C/A3C) — это гибридный подход, объединяющий прямую оптимизацию политики (Actor) и обучение функции ценности (Critic) для снижения дисперсии градиентов и ускорения сходимости. Архитектура состоит из двух нейронных сетей: Actor $\pi_{\theta}(a | s)$ отвечает за выбор действий и обновляется через Policy Gradient, а Critic $V_{\phi}(s)$ обучается предсказывать ожидаемый возврат из состояния и служит baseline для Actor. Вместо использования полных возвратов G_t , Actor оптимизируется по функции преимущества $A(s, a) = Q(s, a) - V(s)$, которая оценивается через одношаговую TD-ошибку: $A(s, a) \approx r + \gamma V(s') - V(s)$. Это показывает, насколько выбранное действие лучше «среднего» для данного состояния.

Обновление происходит параллельно: Critic минимизирует MSE-ошибку предсказания

$L_{critic} = (V(s) - [r + \gamma V(s')])^2$, а Actor делает шаг в направлении

$\nabla \mathcal{J}_{actor} \approx \sum \nabla \log \pi_{\theta}(a | s) A(s, a)$. Согласно лекции, такая схема обладает важными практическими преимуществами: ошибки Critic менее критичны, чем в чистом Q-learning (Actor продолжает обучаться даже при шумной оценке), метод эффективно работает с n-step возвратами для баланса смещения/дисперсии, требует параллельных игровых сессий для стабильности и часто дополняется энтропийной регуляризацией для предотвращения преждевременной сходимости к детерминированным субоптимальным стратегиям. В частично наблюдаемых средах архитектура легко расширяется до LSTM-политик без изменения математического ядра.

Вопросы из недели 7:

1. Применение RNN для генерации последовательностей

Для задач генерации последовательностей (машинный перевод, подписи к изображениям, графем-в-фонему, диалоговые системы) в лекции рассматривается архитектура Encoder-Decoder RNN. Энкодер считывает входные данные (последовательность или произвольный объект) и формирует скрытое состояние, а декодер пошагово генерирует выходную последовательность. Ключевая особенность таких моделей заключается в фундаментальном различии между режимами обучения и инференса. Во время обучения декодер предсказывает следующий токен на основе эталонной (reference) последовательности, минимизируя кросс-энтропию. При инференсе же модель работает в режиме сэмплирования или жадного выбора (max), подавая на вход свои собственные предыдущие предсказания.

Это различие порождает критическую проблему distribution shift: распределение последовательностей, генерируемых моделью во время инференса, отличается от распределения эталонных данных, на которых происходило обучение. Если модель хотя бы раз сгенерирует вариант, отсутствующий в обучающих данных, её внутренние скрытые состояния смещаются, что приводит к накоплению ошибок и нестабильности на следующих шагах. Кроме того, в задачах генерации часто существует несколько корректных вариантов ответа (например, разные формулировки перевода), и обучение только на максимизации правдоподобия эталонных пар может заставлять модель усреднять вероятности, выдавая «мусорные» или излишне осторожные последовательности. Именно эти ограничения supervised-подхода мотивируют переход к reinforcement learning для прямой оптимизации качества генерации.

2. Применение RL к дообучению моделей генерации последовательностей

Применение reinforcement learning позволяет решать проблемы, возникающие при обучении с учителем. Общая формализация RL: максимизировать по политике π ожидаемую награду

$$\mathcal{J} = E_{s \sim d(s), a \sim \pi(a | obs(s))} [R(s, a)], \text{ где } R(s, a) - \text{«чёрный ящик»}, \text{ в частном случае } G(s, a) = r(s, a) + \gamma G(s', a').$$

Основной метод — policy gradient: $\nabla \mathcal{J} = E[\nabla \log \pi_{\theta}(a | s) \cdot Q(s, a)]$. В отличие от supervised learning, где градиент считается по референсным действиям ($\nabla \log \pi_{\theta}(a_{opt} | s)$), RL обучается на собственных сэмплированных действиях модели, что устраняет distribution shift, но требует наличия функции награды и сталкивается с проблемой «холодного старта» и высокой дисперсией оценок.

Для снижения дисперсии применяется Self-Critical Sequence Training (SCST): в качестве базлайна используется награда от greedy-инференса той же модели,

$A(s, a) = R(s, a) - R(s, a_{inference}(s))$. Это позволяет сравнивать сэмплированную последовательность с детерминированной, уменьшая шум градиента. На практике: сначала проводится pre-training через максимизацию $\log P(y | x)$, затем fine-tuning через максимизацию ожидаемой метрики (например, CIDEr для image captioning) с использованием SCST.

В контексте больших языковых моделей применяется RLHF (Reinforcement Learning from Human Feedback):

- 1) Supervised Fine-Tuning на размеченных инструкциях;
- 2) обучение Reward Model на предпочтениях людей (сравнение пар ответов);
- 3) дообучение политики через PPO с наградой от модели и регуляризацией через исходный supervised loss. Альтернатива — Direct Preference Optimization (DPO), которая упрощает процесс, оптимизируя лог-правдоподобие предпочтений напрямую, без явного обучения reward model и использования PPO.